

Week 5 · Student Notes

AI531C Digital Image Analysis · Tue 1 – Wed 2 Sep 2026 · Unit II · Lectures 11, 12, 13

Everything taught in this week, in one document. The per-lecture handouts are identical in content — use whichever you prefer.

What is in this bundle

- Lecture 11 — Spatial Denoising and Noise-Specific Filter Selection
- Lecture 12 — Inverse, Wiener and Regularised Restoration
- Lecture 13 — Geometric Transformations and Interpolation

This week at a glance

Sessions — Tue 1 – Wed 2 Sep 2026.

Tutorial — T5 (Tue 1 Sep).

Pre-class preparation — PCM-5.

Released after the last class of the week: Wed 2 Sep 2026, 18:00.

How to use this bundle — read the lecture section on the evening of the class while the argument is fresh, attempt the check-your-understanding questions before the tutorial, and leave the practice questions until you can do them unaided. No answer keys are printed: the reasoning is worked through in the tutorial.

Lecture 11 · Spatial Denoising and Noise-Specific Filter Selection

AI531C Digital Image Analysis · Unit II · CO2 · M.Tech. Artificial Intelligence, Department of CSE, DTU

The L8 filter zoo returns — this time with ground truth, PSNR evidence, and a verdict per noise type

What you should already know

This lecture continues from Lecture 10, “The Image Degradation/Restoration Model and Noise Processes”. Revise that material first — today's arguments assume it.

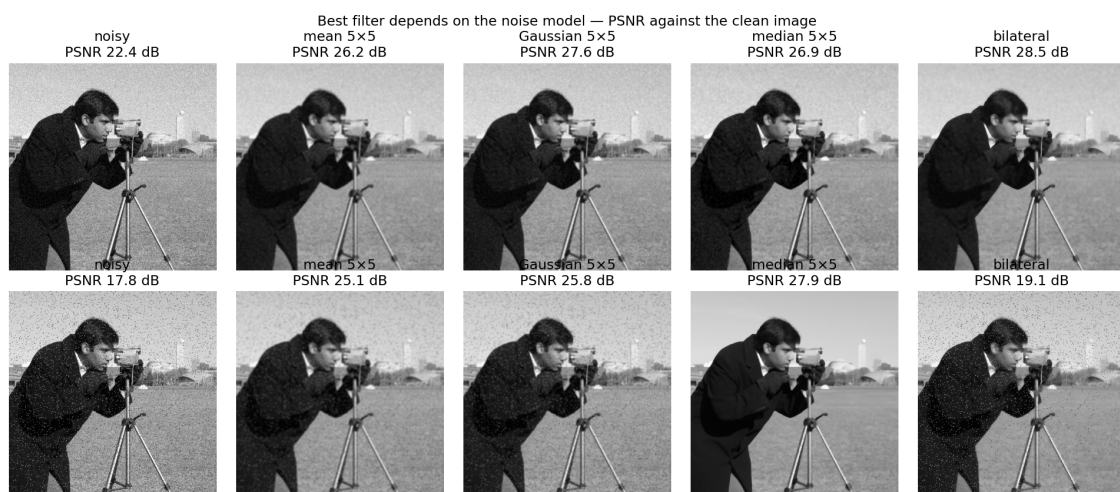
Reading in the course notes: Comprehensive Notes L11; Vol. I ch. 11.

Learning outcomes

After studying this lecture you should be able to:

- match mean, Gaussian, median, bilateral and adaptive filters to noise models.
- read a PSNR table and state what it does and does not measure.
- explain WHY the median defeats impulses and the bilateral preserves edges.
- quantify the edge-preservation trade-off.
- design a denoising comparison experiment.

Figure 1: The evidence: filter × noise, with PSNR



Two noise types × four filters, PSNR vs clean reference. Generated by L11_denoising_comparison.py.

How to read this figure

What to notice — Gaussian row: bilateral wins (28.5 dB). Impulse row: median wins (27.9 dB) and bilateral COLLAPSES (19.1 dB — barely above the noisy 17.8).

Why it happens — Bilateral's range weight trusts similar pixels — impulses are never similar to anything, but they poison the local average anyway.

What it means — No filter is best; each is best FOR a degradation.

Why it matters — L10's identification step is not optional bureaucracy — it decides 9 dB.

Three filters, three philosophies

Problem: estimate the true value at (x, y) from a corrupted neighbourhood S . Assume the noise model is known from L10's identification step.

$$\text{mean: } \hat{g} = (1/|S|) \sum f \quad \text{median: } \hat{g} = \text{med}\{f \in S\} \quad \text{bilateral: } \hat{g} \propto \sum e^{-\|p-q\|^2/2\sigma_s^2} e^{-\{(f_p-f_q)^2/2\sigma_r^2\}} f_q$$

What each symbol means

- S — neighbourhood; $|S|$ — its size σ_s — spatial scale: how far to look σ_r — range scale: how different is 'different' med — order statistic (rank, not sum)

How to read this

What it means — Mean averages everything (linear); median votes (robust to outliers); bilateral averages only pixels that are BOTH near and similar — a Gaussian blur that refuses to cross edges. Adaptive filters modulate strength by local variance: strong in flat areas, gentle on detail.

Worked example

Given — 3×3 neighbourhood [12, 14, 13; 11, 255, 12; 13, 12, 14] — a salt impulse (255) at the centre of a flat ~ 12 -level region.

Find — The 3×3 mean-filter and median-filter outputs at the centre.

Formula — mean = $\Sigma/9$; median = 5th of 9 sorted values

Solution

Substitution — $\Sigma = 356$; sorted: 11, 12, 12, 12, 13, 13, 14, 14, 255

Calculation — mean = $356/9 = 39.6$; median = 13 ANSWER: mean ≈ 40 (impulse smeared in); median = 13 (impulse erased)

What it means — The mean gives the outlier 1/9 of the vote forever; the median gives it none unless outliers are the majority. One arithmetic line explains the whole PSNR table.

The code behind the results

```
def psnr(ref, x): mse = np.mean((ref - x)**2)
return 10*np.log10(255**2/mse)
for name, f in filters.items(): out = f(noisy)
print(name, psnr(clean, out))
```

Parameters used — fixed rng seed; identical noisy input per filter. Full program:

L11_denoising_comparison.py

What it means — A fair comparison = same input, same reference, same metric.

Something to think about — No clean reference? Flat-patch σ and visual panels take over (T5 practises both).

The verdict table

Filter	Gaussian noise	Impulse noise	Edge cost
Mean 5×5	26.2 dB — works, blurs	25.1 dB — smears impulses	high
Gaussian 5×5	27.6 dB — solid default	25.8 dB — same weakness	moderate
Median 5×5	26.9 dB — decent	27.9 dB — the specialist	low on thin edges, eats texture
Bilateral	28.5 dB — best here	19.1 dB — collapses	lowest (by design)

Key takeaway: these numbers are from ONE image at ONE noise level — the method (identify → benchmark → verdict) generalises; the numbers do not.

Where this is used — your project

Denoising lane — this lecture IS your baseline section: identified noise + benchmark harness + verdict table on YOUR data (T5 artefact, reused at T8, graded against your CNN at T11);

Every lane — denoising is usually a preprocessing stage — record filter + parameters in the pipeline log, and re-run downstream metrics with and without it: preprocessing must EARN its place.

Try this yourself — Median vs bilateral on your noisiest project image; keep the loser's output too — failure evidence is portfolio material.

Summary

- Filter choice follows the noise model — the difference is measured in dB, not taste. Median = voting (robust); bilateral = similarity-gated averaging (edge-keeping); each has a failure regime. A fair benchmark needs same input, same reference, same metric — and honesty when no reference exists.

Key formulae

$PSNR = 10 \log_{10}(255^2/MSE)$ bilateral weight $\propto e^{-d^2/2\sigma_s^2} \cdot e^{-\Delta f^2/2\sigma_r^2}$ median fails when outliers $\geq \lceil |S|/2 \rceil$

What comes next

Tutorial T5 — today 2:00–3:00 PM (studio) — Noise & restoration studio: simulate L10's noise types on your own images, run the L11 benchmark harness, produce the metric table + output grid + one failure note (lesson-plan artefact; feeds the project data-readiness checkpoint).

Before tomorrow (L12–L13 double lecture) — Re-read the L9 convolution theorem summary — L12 works entirely in $G = H \cdot F + N$.

Logistics — L12 (Wed 10 AM): inverse, Wiener, regularised restoration. L13 (Wed 11 AM): geometric transforms & interpolation. PCM-6 releases Fri.

Check your understanding

Work these before the tutorial. They are the same questions used in class; the reasoning is discussed there, so attempt them first.

1. Parameter prediction: the bilateral dials

A portrait has mild Gaussian noise. You run a bilateral filter three times: A. $\sigma_r = 10$ (tiny range scale) B. $\sigma_r = 60$ (moderate) C. $\sigma_r = 500$ (huge) Predict each output in one phrase, then rank them for (i) denoising strength and (ii) edge preservation.

Practice questions

Attempt these unaided, then compare with the discussion in your tutorial session.

Q1 (4 marks, understand): Explain, via order statistics, why a median filter removes salt-and-pepper noise that defeats a Gaussian filter of the same size.

Q2 (6 marks, apply): For the neighbourhood [20, 22, 21; 19, 0, 20; 22, 21, 20] (a pepper impulse), compute the 3×3 mean and median outputs and the absolute error of each against the local level ≈ 20.5 .

Q3 (10 marks, analyse): A drone survey team applies a 7×7 median to all imagery 'for safety' before crack detection, and thin cracks disappear from results. Diagnose, quantify the mechanism, and redesign the pipeline.

Further reading

GW4 Ch.5 Secs.5.3–5.4; Szeliski neighbourhood operators

Course notes: Comprehensive Notes L11; Vol. I ch. 11.

Runnable code for every figure and number in this lecture: 06_CODE_AND_DEMOS (see the file names quoted beside each figure).

AI531C Digital Image Analysis · student handout · every figure and numerical result in this document is the output of course code you can run yourself.

Lecture 12 · Inverse, Wiener and Regularised Restoration

AI531C Digital Image Analysis · Unit II · CO2 · M.Tech. Artificial Intelligence, Department of CSE, DTU

*Undoing blur is division — and division by almost-zero is how restoration destroys itself.
Regularisation is the cure*

What you should already know

This lecture continues from Lecture 11, “Spatial Denoising and Noise-Specific Filter Selection”.
Revise that material first — today's arguments assume it.

Reading in the course notes: Comprehensive Notes L12; Vol. I ch. 12.

Learning outcomes

After studying this lecture you should be able to:

- explain why deblurring is ill-posed.
- derive the inverse filter and show where it explodes.
- state the Wiener filter and interpret its K term.
- read regularised restoration as a data-fit + smoothness trade-off.
- choose a restoration strategy from what is known about h and η .

Restoration as division — and why that is dangerous

Problem: given the observed G and known blur H , recover F . Model (L10): $G = H \cdot F + N$. Assume h known (non-blind restoration); noise statistics estimable (L10's flat patch).

$$F_{inv} = G / H = F + N / H$$

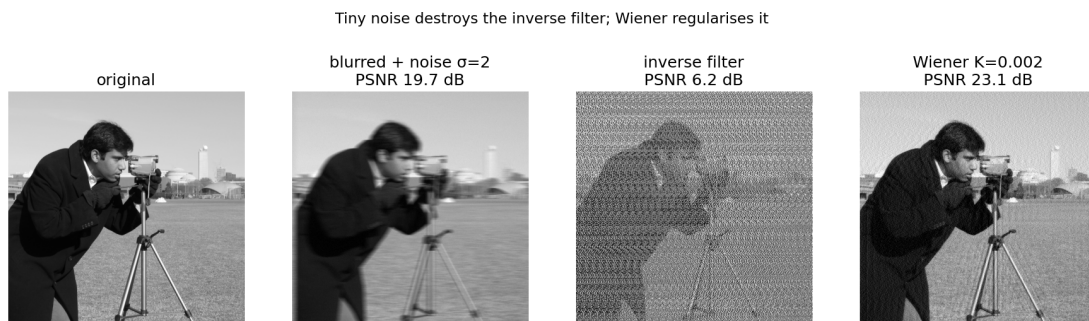
What each symbol means

- F — estimate of the true spectrum H — blur spectrum (OTF): near ZERO at high frequencies for any real blur N/H — the noise, divided by almost-nothing The second equality: substitute G and cancel

How to read this

What it means — The algebra is exact and the result is useless: wherever $|H| \approx 0$, the noise term N/H explodes — tiny noise becomes the whole image. Measured on the panel: $\sigma = 2$ noise (invisible!) drives the inverse result to 6.2 dB — worse than the blurred input at 19.7 dB.

Figure 1: The catastrophe, measured



Motion blur (15 px) + $\sigma=2$ noise: inverse vs Wiener. Generated by L12_inverse_wiener.py.

How to read this figure

What to notice — Blurred input: 19.7 dB. Inverse filter: 6.2 dB of structured garbage. Wiener ($K = 0.002$): 23.1 dB — better than the input.

Why it happens — Motion blur's H has periodic near-zeros; N/H erupts exactly there — the streaky texture IS amplified noise at those frequencies.

What it means — More model (knowing noise exists) beats more algebra (exact inversion).

Why it matters — Never deploy an inverse filter on real data. Ever.

Worked example

Given — At some frequency, $|H| = 0.01$ (deep in a blur null). Noise-to-signal constant $K = 0.002$. Real-valued H for simplicity.

Find — The inverse-filter gain and the Wiener gain at this frequency.

Formula — inverse: $1/|H|$; Wiener: $|H| / (|H|^2 + K)$

Solution

Substitution — inverse = $1/0.01$; Wiener = $0.01 / (0.0001 + 0.002)$

Calculation — inverse = 100; Wiener = $0.01 / 0.0021 = 4.76$ ANSWER: inverse amplifies 100×; Wiener only 4.8×

What it means — At this frequency the signal is 100× attenuated but the noise is not — the inverse restores both equally; Wiener weighs the evidence and mostly declines. 21× less amplification is the 17 dB difference in the panel.

Core concepts

The regularisation view (Tikhonov)

The estimator as an optimisation — $\hat{f} = \operatorname{argmin}_f \|Hf - g\|^2 + \lambda \|L f\|^2$ — fit the data, but penalise solutions that are rough (L = gradient/Laplacian operator).

What λ does — $\lambda \rightarrow 0$: pure data fit $\rightarrow$ inverse filter and its explosion. λ large: heavy smoothing $\rightarrow$ blur returns. λ is the SAME dial as Wiener's K , in optimisation clothing.

Why this framing matters — Swap the penalty $\|Lf\|^2$ for a learned prior and this exact equation becomes modern restoration — Unit IV's networks are, in one honest sentence, regularisers learned from data. The bridge to L27–L30 starts here.

The code behind the results

```
H = np.fft.fft2(psf)
# known blur Gn = np.fft.fft2(g) K = 0.002 # from sigma W = np.conj(H) / (np.abs(H)**2 + K)
f_hat = np.real(np.fft.ifft2(W * Gn))
```

Parameters used — K sweeps 10^{-4} ... 10^{-1} in the studio. Full program: L12_inverse_wiener.py

What it means — $\operatorname{conj}(H)$ handles complex H (asymmetric PSFs); K is the honesty dial.

Something to think about — PSF length 13 vs true 15? Ringing — model error acts like structured noise (T5/T6).

Where this is used — your project

Deblurring lane — Wiener with measured PSF is your REQUIRED classical baseline (L31's learned deblurring is graded against it); document PSF estimation method + K ;

Denoising / low-light lanes — the λ -dial concept transfers: every method has an aggressiveness parameter — sweep it and SHOW the trade-off curve, don't pick one silently;

All lanes — 're-acquire beats restore' — one slide of acquisition improvement discussion is worth marks in every rubric.

Try this yourself — Wiener K sweep $\{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$ on the panel's data; plot PSNR vs K — find the peak.

Summary

- Exact inversion + real noise = catastrophe; ill-posedness is measurable (19.7 → 6.2 dB). Wiener = inverse where evidence is strong, silence where it is not — K is the honesty dial. Data-fit + prior is the master equation: classical priors today, learned priors in Unit IV.

Key formulae

$$\hat{F}_{inv} = G/H = F + N/H \quad W = H^*/(|H|^2 + K) \quad \hat{f} = \operatorname{argmin} \|Hf - g\|^2 + \lambda \|Lf\|^2$$

What comes next

Immediately next (L13, same room) — Geometric transformations and interpolation — the machinery beneath every resize you have run since L2.

PCM-6 (releases Fri 4 Sep, due Mon 7 Sep, ~40 min) — Registration scavenger: photograph two views of a planar scene, mark ≥ 6 correspondences — becomes YOUR data in tutorial T6.

Logistics — T6 (Tue 8 Sep): geometry & registration studio on your own image pairs. L14 (Tue 8 Sep): registration, alignment, homography.

Check your understanding

Work these before the tutorial. They are the same questions used in class; the reasoning is discussed there, so attempt them first.

1. Which restoration strategy?

For each case, choose: inverse / Wiener / regularised / 'don't restore, re-acquire': A. Microscope with a calibrated, gentle defocus PSF; negligible noise. B. CCTV frame, strong motion blur, visible noise, PSF estimated roughly. C. One-off historical photograph, unknown blur. D. Industrial line where a lens can simply be refocused.

Practice questions

Attempt these unaided, then compare with the discussion in your tutorial session.

Q1 (4 marks, understand): Why is deblurring ill-posed even when the blur h is known exactly? Answer via the behaviour of H at high frequencies.

Q2 (6 marks, apply): At a frequency with $|H| = 0.05$ and $K = 0.001$, compute inverse and Wiener gains and the amplification ratio between them.

Q3 (10 marks, analyse): A student reports 'Wiener made my image WORSE than the blurry input' with $K = 10^{-6}$ on noisy data. Diagnose, connect to the inverse filter, and prescribe a principled way to set K .

Further reading

GW4 Ch.5 Secs.5.5-5.9; Szeliski Ch.3 Sec.3.7

Course notes: Comprehensive Notes L12; Vol. I ch. 12.

Runnable code for every figure and number in this lecture: 06_CODE_AND_DEMOS (see the file names quoted beside each figure).

AI531C Digital Image Analysis · student handout · every figure and numerical result in this document is the output of course code you can run yourself.

Lecture 13 · Geometric Transformations and Interpolation

AI531C Digital Image Analysis · Unit II · CO2 · M.Tech. Artificial Intelligence, Department of CSE, DTU

Moving pixels instead of changing them — and the interpolation question every move forces you to answer

What you should already know

This lecture continues from Lecture 12, “Inverse, Wiener and Regularised Restoration”. Revise that material first — today's arguments assume it.

Reading in the course notes: Comprehensive Notes L13; Vol. I ch. 13.

Learning outcomes

After studying this lecture you should be able to:

- write translation, rotation, scaling, affine and projective maps in homogeneous coordinates.
- explain backward mapping and why it needs interpolation.
- compute a bilinear interpolation by hand.
- choose NN / bilinear / bicubic per task (masks vs images!).
- count the degrees of freedom of each transform family.

The transform hierarchy in homogeneous coordinates

Problem: map pixel positions (x, y) to new positions (x', y') with one matrix operation. Homogeneous trick: append 1, so translation becomes linear too.

$$[x', y', w]^T = T \cdot [x, y, 1]^T, \text{ then divide by } w$$

What each symbol means

- T — 3×3 transform matrix; w — homogeneous coordinate
- Rigid: rotation + translation (3 DoF)
- Affine: + scale/shear (6 DoF, last row $[0 \ 0 \ 1]$, $w = 1$)
- Projective/homography: full 3×3 (8 DoF, w varies)

How to read this

What it means — Each family preserves less: rigid keeps lengths; affine keeps parallelism; projective keeps only straight lines. DoF count = the number of point correspondences needed to estimate it ($\div 2$) — the fact L14's RANSAC lives on.

Core concepts

Backward mapping: why warping needs interpolation

The naive (forward) way fails — Pushing each source pixel to its target leaves holes and collisions — rotated grids do not land on grids.

The real (backward) way — For every OUTPUT pixel, apply T^{-1} to find where it came from in the source — a non-integer position — then interpolate the source there. Every warpAffine/warpPerspective call does exactly this.

So the quality question becomes — how do we read an image between its pixels? Nearest neighbour (copy the closest), bilinear (weight the 4 surrounding), bicubic (fit the 16 surrounding). The next two slides make this concrete and visible.

Worked example

Given — Backward mapping lands at source position (row $y = 0.25$, col $x = 0.75$) between four pixels: $f(0,0)=10$, $f(0,1)=20$, $f(1,0)=30$, $f(1,1)=40$.

Find — The bilinearly interpolated value.

Formula — $f = (1-y)(1-x) \cdot f_{00} + (1-y)x \cdot f_{01} + y(1-x) \cdot f_{10} + yx \cdot f_{11}$

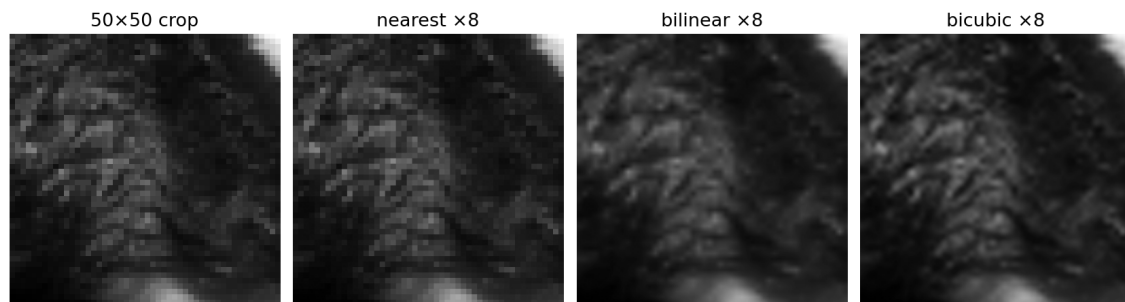
Solution

Substitution — $(0.75)(0.25) \cdot 10 + (0.75)(0.75) \cdot 20 + (0.25)(0.25) \cdot 30 + (0.25)(0.75) \cdot 40$

Calculation — $1.875 + 11.25 + 1.875 + 7.5$ ANSWER: $f = 22.5$

What it means — Weights are the OPPOSITE-corner areas and sum to 1 — a convex combination: bilinear can never overshoot its four neighbours (bicubic can, by design).

Figure 1: Interpolation quality, magnified ×8



50×50 face crop enlarged ×8 by three interpolators. Generated by L13_geometric_interpolation.py.

How to read this figure

What to notice — NN: visible blocks. Bilinear: smooth but soft. Bicubic: smoothest gradients with slightly crisper edges.

Why it happens — NN copies (piecewise constant); bilinear fits planes (C^0); bicubic fits cubics (C^1 , mild overshoot at edges).

What it means — Interpolation order = smoothness assumption about the scene between samples.

Why it matters — Images: bilinear/bicubic. MASKS: NN only — averaging class IDs invents classes (L3).

The code behind the results

```
M = cv2.getRotationMatrix2D(centre, 20, 0.9)
out = cv2.warpAffine(img, M, (w, h), flags=cv2.INTER_LINEAR) P =
cv2.getPerspectiveTransform(src4, dst4)
out2 = cv2.warpPerspective(img, P, (w,h))
```

Parameters used — interpolation flag; 4 point pairs for perspective. Full program: L13_geometric_interpolation.py

What it means — getPerspectiveTransform solves the 8 DoF exactly from 4 correspondences.

Something to think about — The 4 points slightly wrong? Confidently wrong everywhere — L14: many points + RANSAC.

Interpolation selection table

Method	Reads	Character	Use for
Nearest	1 px	blocky, exact values preserved	label masks, indexed images, speed
Bilinear	4 px	smooth, no overshoot, slightly soft	general resizing/warping default
Bicubic	16 px	smoothest, mild overshoot/ringing	quality upscaling of photographs
Area	varies	averages the footprint (anti-alias)	DOWNSampling (the L3 rule)

Key takeaway: one flag argument, four different claims about your data — set it deliberately in every resize and warp.

Where this is used — your project

Change-detection lane — both dates must be resampled ONCE, with a stated interpolation, onto one grid — double-resampling smears evidence; document the chain;

Super-resolution lane — bicubic upscaling is your mandatory baseline (L32 is graded against it) — implement it today, not in October;

Segmentation lane — augmentation rotations/scales warp BOTH image (bilinear) and mask (NN) — mismatched flags silently corrupt training labels (L25 formalises).

Try this yourself — Warp any mask with bilinear, then NN; difference the results and count corrupted pixels.

Summary

- Homogeneous coordinates make every move one matrix; DoF counts the correspondences needed. Warping = backward mapping + interpolation: a sampling question in disguise. Interpolation is a smoothness claim — masks are not smooth; NN protects them.

Key formulae

$[x', y', w]^T = T[x, y, 1]^T$, divide by w bilinear: opposite-area weights, $\Sigma = 1$ affine 6 DoF / 3 pts; homography 8 DoF / 4 pts

What comes next

PCM-6 (releases Fri 4 Sep, due Mon 7 Sep, ~40 min) — Registration scavenger: two views of a planar scene with small viewpoint change; mark ≥ 6 correspondences and note likely mismatches — your data for T6.

Tuesday 8 Sep — L14 (12 PM): registration, alignment, homography. T6 (2–3 PM): geometry & registration studio on your own pairs.

Logistics — Bring both photos digitally to T6. Planar, textured scenes work best (posters, façades).

Check your understanding

Work these before the tutorial. They are the same questions used in class; the reasoning is discussed there, so attempt them first.

1. Predict: the coordinate trip

An affine transform doubles x , halves y , then translates by $(+10, +20)$: $T = \begin{bmatrix} 2 & 0 & 10 \\ 0 & 0.5 & 20 \\ 0 & 0 & 1 \end{bmatrix}$ 1. Where does pixel $(x, y) = (5, 8)$ land? 2. An output pixel sits at $(30, 22)$ — where must backward mapping read the source? 3. Is that source position readable directly, or does it need interpolation?

Practice questions

Attempt these unaided, then compare with the discussion in your tutorial session.

Q1 (4 marks, understand): Why does image warping use backward rather than forward mapping? Name the two artefacts forward mapping produces.

Q2 (6 marks, apply): Compute the bilinear value at $(y, x) = (0.5, 0.25)$ for corners $f_{00}=40, f_{01}=80, f_{10}=20, f_{11}=60$, and state the DoF and minimum correspondences for an affine map.

Q3 (10 marks, analyse): After augmenting a segmentation dataset with 15° rotations, a team finds thin 'ghost' class boundaries in training masks and falling validation accuracy. Diagnose the mechanism and give the correct augmentation recipe.

Further reading

GW4 Ch.2 interpolation; Szeliski Ch.3 Sec.3.6; Forsyth-Ponce geometry

Course notes: Comprehensive Notes L13; Vol. I ch. 13.

Runnable code for every figure and number in this lecture: 06_CODE_AND_DEMOS (see the file names quoted beside each figure).

AI531C Digital Image Analysis · student handout · every figure and numerical result in this document is the output of course code you can run yourself.